Ship Without Engineers — The Director Model for Building Software Products

50+	10min	$0	6wks
PRODUCTS SHIPPED	PER PROTOTYPE	ENGINEERING FEES PAID	TO FIRST REVENUE

Why This Exists

You have the idea. You have the customers. You don't have a CTO.

This is not a guide about learning to code. It is a guide about learning to direct. There is a real difference. A director does not write the screenplay. A director specifies what they want, reviews the output, and tells the crew what to change. That is exactly what you will do with Claude Code.

Everything in this guide came from building 50+ working products in 6 weeks with no CS degree and no prior programming experience. Apps, landing pages, automations, digital products. The system works. You can run it from day one.

> **What you get:** The director model for giving precise instructions. The 10-minute prototype workflow. The decision framework for what to build first. Step-by-step deploy commands for Surge, Vercel, Gumroad, and the App Store. The 5-question non-technical review. The product catalog system for tracking 50+ products without losing your mind.

The Core Insight

Non-technical founders fail with AI coding tools for one reason: they ask vague questions and get vague output. "Build me a landing page" produces generic garbage. "Build me a single-file HTML landing page for a $47 pricing calculator tool, dark background, monospace font, two-column layout with a form on the left and live output on the right, no external dependencies" produces something you can deploy in 10 minutes.

Precision is the skill. Not code. Precision.

System Stats (Real, Not Projected)

Metric	Value
Products built	50+
Engineering fees paid	$0
Time from idea to deployed prototype	Under 10 minutes
Weeks to first revenue	6
Uptime on deployed products	99%+
Platforms used	Surge.sh, Vercel, Gumroad, App Store
Build types	HTML tools, landing pages, PWAs, PDFs, calculators

Who This Is For

You have built a business before but not software products

You understand customers, pricing, and distribution. You just need the production layer. This guide closes that gap.

EXACT FIT

You want to move at engineer speed without hiring engineers

A freelance developer costs $75–150/hr and takes 2 weeks to onboard. Claude Code is immediate. With the right spec, the output is indistinguishable from paid work for most product types.

EXACT FIT

You have Claude Code but don't feel confident directing it

The tool is there. The missing piece is the instruction framework. That's what this guide provides.

EXACT FIT

The Director Model

Your job is not to write code. Your job is to specify clearly, review outputs, and direct revisions. That is it. The entire model collapses to three verbs: specify, review, revise.

> **Director vs Engineer:** An engineer decides how to build something. A director decides what to build and whether the result matches the vision. You are the director. Claude Code is the crew. Never confuse the roles.

The 4 Types of Instructions

Every instruction you give Claude Code falls into one of four categories. Master these and you can direct any build.

1. Feature Spec

Describes exactly what a feature does. Includes the trigger, the action, and the result.

MOST IMPORTANT

```
"Add a button labeled 'Calculate' that, when clicked, reads the
value from the #revenue input field, multiplies it by the margin
percentage in #margin, and displays the result in #output with
two decimal places and a dollar sign. Save the result to
localStorage under the key 'last_calculation'."
```

2. Constraint

Tells Claude Code what NOT to use or do. Prevents scope creep and dependency problems.

SCOPE CONTROL

```
"No external libraries. No API calls. No npm packages.
Everything must work offline by opening the HTML file
directly in a browser. No build step required."
```

3. Style Reference

Points to an existing example instead of describing aesthetics in words. Always faster and more accurate.

FASTER THAN DESCRIBING

```
"Dark background (#0a0a0f), monospace font (SF Mono or Fira Code),
minimal UI with no decorative elements. Use the card pattern
from financial_dashboard.html as the reference for spacing and
border treatment."
```

4. Scope Boundary

Tells Claude Code exactly which part of the file to touch and which parts to leave alone.

PREVENTS BREAKAGE

```
"Only modify the <header> section and the .nav CSS block.
Do not touch any JavaScript. Do not change the body layout.
Do not add new HTML elements outside the header."
```

The 5 Words That Save You

When you have an existing example and want something similar:

> **"Exactly like the example, except..."**
> This single phrase eliminates 80% of miscommunication. You point at something that already works, then describe only the delta. Claude Code fills in everything else from the reference.

What NOT to Do

> **Never say "make it better."** Better at what? Faster? Prettier? More accurate? Shorter? Without a specific criterion, Claude Code will make arbitrary changes that may break things that already worked. Always define the direction of improvement before asking for it.

The Revision Cycle

Every build follows this loop:

```
1. SPEC    Write a precise CLAUDE.md describing what you want
2. BUILD   Run: claude --dangerously-skip-permissions
3. REVIEW  Use the 5-question framework (see Review section)
```

```
4. REVISE   Describe what is wrong specifically, not "fix it"
5. SHIP     When 5/5 questions pass, deploy
```

Good Spec vs Bad Spec

The single biggest variable in output quality is spec quality.

Bad CLAUDE.md Spec

Vague. No constraints. No reference. No scope.

```
Build a pricing tool for my SaaS business.
Make it look professional.
Include pricing tiers.
```

Good CLAUDE.md Spec

Specific. Constrained. Referenced. Scoped.

```
# Product: SaaS Pricing Calculator

## What it does
- User enters monthly active users (number input, 1-100000)
- User selects tier (radio: Starter / Growth / Enterprise)
- Tool displays monthly price and annual price with 20% discount
- Displays a breakdown: base fee + per-seat fee

## What it does NOT do
- No sign-up form
- No payment processing
- No external API calls

## Tech constraints
- Single HTML file, self-contained
- No external libraries or CDN links
- Works offline (open file:// in browser)
- Mobile-responsive

## Style
- Dark background: #0a0a0f
```

```
- Monospace font: 'SF Mono', 'Fira Code', monospace
- Reference: financial_dashboard.html card style

## Scope
- Create ONE new file: pricing_calculator.html
- Do not modify any existing files
```

The Build Workflow

Idea to working prototype in under 10 minutes. This is the exact process used to ship 50+ products.

The 10-Minute Prototype

```
Step 1 (2 min): Write your spec in CLAUDE.md using the template below
Step 2 (1 min): Open terminal, navigate to your project folder
Step 3 (5 min): Run: claude --dangerously-skip-permissions
                Claude Code reads CLAUDE.md and builds autonomously
Step 4 (2 min): Open the output file in your browser, run the 5-question review
```

> **--dangerously-skip-permissions** lets Claude Code write files and run commands without asking for permission on each step. This is what makes autonomous builds possible. It is safe in your project folder. Do not run it in directories you don't fully control.

The Spec Template (Fill In and Use)

Copy this into a file called `CLAUDE.md` in your project folder. Fill in the brackets.

```
# Product: [Name of the product]

## One-sentence description
[What it does for the user in one sentence]
```

```
## What it does (3 bullet points max)
- [Core feature 1]
- [Core feature 2]
- [Core feature 3]

## What it does NOT do (2 bullet points max)
- [Out-of-scope feature 1 — prevents scope creep]
- [Out-of-scope feature 2]

## Tech constraints
- Single HTML file, self-contained, no external dependencies
- No build step. Open directly in browser.
- Mobile-responsive (test at 375px width)
- [Add any other hard constraints here]

## Style
- Background: #0a0a0f (dark)
- Font: monospace (SF Mono, Fira Code, JetBrains Mono)
- [Reference file if you have one, e.g.: "reference: dashboard.html"]

## Output
- Create ONE file: [filename].html
- Do not modify any existing files
- Do not create any other files
```

3 Prototype Categories

HTML Tools

Self-contained single-file apps that do one specific calculation or task. No server. No database. Works offline. Highest value-to-build-time ratio.

FASTEST TO BUILD | HIGHEST MARGIN

```
Examples:
- Financial dashboard (revenue, margin, burn rate inputs)
- ROI estimator for a specific industry
- Pricing calculator for a SaaS product
- Email template generator with variable substitution
- Proposal cost calculator
```

Landing Pages

Single HTML file with hero, benefits, social proof, and CTA. Deploy to Surge.sh in 30 seconds. Use for waitlists, digital products, ebook pre-sales, tool launches.

30-SEC DEPLOY

```
Examples:
- Course waiting list (email capture + Mailchimp)
- Ebook landing page (buy on Gumroad, preview on page)
- Tool waitlist (coming soon with email capture)
- Service landing page for local business
```

PDF Content Products

Write in markdown, export to PDF, sell on Gumroad. Claude Code can generate the markdown content and convert it to a polished HTML version for the guide file itself.

LOWEST EFFORT

```
Examples:
- Industry-specific checklist
- Swipe file (templates + examples)
- Decision framework with worked examples
- Resource directory with commentary
```

Full Example Spec: Pricing Calculator

This exact spec was used to build a working tool in 7 minutes.

```
# Product: Agency Retainer Calculator

## One-sentence description
Helps agency owners calculate the correct monthly retainer
```

```
based on hours, team size, overhead, and target margin.

## What it does
- Accepts: estimated hours/month, hourly cost, overhead percentage,
  target profit margin percentage
- Calculates: break-even rate, recommended retainer, annual value
- Displays: color-coded output (green = healthy margin, red = below break-even)

## What it does NOT do
- No client management features
- No invoice generation
- No saving or accounts

## Tech constraints
- Single HTML file, self-contained
- No external libraries
- Works offline
- Mobile responsive at 375px

## Style
- Background: #0a0a0f
- Font: 'SF Mono', monospace
- Cards for input and output sections
- Green (#22c55e) for positive outputs, red (#ef4444) for warnings

## Output
- Create: agency_retainer_calculator.html
- Do not modify any other files
```

What to Build First

You have 100 ideas. The wrong choice costs 3 weeks. The right choice gets you to revenue in week one. Use this framework to sort them.

The Venture Scoring Matrix

Score each idea on three dimensions, multiply, divide by competition. Highest score builds first.

> **Score = (Market Size x Monetization Ease x Build Speed) / Competition Level**
> Rate each dimension 1-5. Market Size: how many people have this problem. Monetization Ease: how directly does it convert to money. Build Speed: how fast can you build it. Competition Level: how crowded is the space (lower is better, so divide).

Tier 1: Build in Week One

HTML tools with immediate, specific utility. These build in under 2 hours and can be sold the same day.

Pricing Calculator

Industry-specific pricing tool. A roofer calculates job cost. A consultant calculates project fee. A SaaS founder calculates MRR scenarios. Niche-specific = high willingness to pay.

WEEK 1

ROI Estimator

Shows a buyer the return on their investment before they buy. Used in B2B sales. Reduces objections. Can be sold to sales teams as a tool or used to generate leads on your own site.

WEEK 1

Email Template Generator

User fills in their company, product, and target customer. Tool generates 5 cold email variants. Sell to sales teams, freelancers, or agency founders. $29-47 per download.

WEEK 1

Tier 2: Build in Weeks 2-3

Landing Page for a Digital Product

Once you have one HTML tool working, build a landing page to sell it. Course waiting lists, ebook pre-sales, tool waitlists. Single HTML file. Deploy to Surge in 30 seconds.

WEEKS 2-3

Lead Magnet Tool

A free tool that captures email before showing the output. Industry-specific enough that it only attracts your exact buyer. Powers your email list while demonstrating your expertise.

WEEKS 2-3

Tier 3: Build in Month 2

Lightweight Apps with User State

Habit trackers, quiz tools, calculators that save progress. These require localStorage or a simple backend. More complexity, but higher perceived value and better retention.

MONTH 2

Build vs. Don't Build Decision Table

Product Type	Build Time	Price Range	Best Platform
HTML calculator tool	Under 2 hours	$19-47	Gumroad
Landing page (single)	Under 1 hour	Free (lead gen)	Surge.sh

PDF guide/checklist	2–4 hours	$9–29	Gumroad
Landing page with waitlist	Under 2 hours	Free (list building)	Surge.sh / Vercel
Habit tracker PWA	4–8 hours	$2.99/mo IAP	App Store
Quiz or assessment tool	2–4 hours	$29–97	Gumroad / own site
SaaS MVP	2–6 weeks	$29–99/mo	Vercel + Stripe

> **The kill criterion before you start:** If you cannot describe what your product does in two clear bullet points in under 5 minutes, the idea is not clear enough yet. Do not build it. Spend 30 more minutes on the spec. A fuzzy idea produces a fuzzy product and fuzzy products do not sell.

> **The 2-hour rule:** If specifying the product (not building it, just writing the spec) takes more than 2 hours, the scope is too large. Split it into two simpler products. Smaller, more specific tools outperform complex ones on Gumroad almost every time.

Deploy

Four options. Pick based on what you built. Each has a one-command deploy path.

Option 1: Surge.sh (HTML/CSS/JS Tools and Landing Pages)

Free. Instant. No account setup required beyond npm install. Works for any static HTML file or folder.

```
# Install once
npm install -g surge

# Deploy a single file
```

```
surge pricing_calculator.html mysite.surge.sh

# Deploy a folder
surge dist/ mysite.surge.sh

# Custom subdomain on your own domain
surge dist/ tools.yourdomain.com
```

> **Surge is the right choice for 90% of HTML tools and landing pages.** Free tier handles unlimited sites. Deploy takes 30 seconds. URL is live immediately. No build step required.

Option 2: Vercel (Next.js or Multi-Page Apps)

Free tier. Works for more complex apps that need routing, API endpoints, or a build step.

```
# Install once
npm install -g vercel

# Deploy from project root
vercel --prod

# Set environment variables
vercel env add STRIPE_SECRET_KEY

# Deploy with specific settings
vercel --prod --name my-app-name
```

Option 3: Gumroad (Digital Products)

For paid guides, HTML tools, and PDFs. Upload the file directly. Gumroad handles payments, delivery, and VAT in most countries.

```
# No command line needed.
# 1. Go to app.gumroad.com
# 2. New Product > Digital Product
# 3. Upload your .html or .pdf file
# 4. Set price (recommended: $39 with PWYW minimum $19)
# 5. Publish. Share the link.
```

```
# The file this guide is in IS the Gumroad product.
# The buyer downloads this HTML file and opens it in any browser.
# No server. No login. No expiry.
```

Option 4: App Store via Capacitor (PWA to iOS App)

Takes your HTML/JS web app and wraps it as a native iOS app. Requires $99/year Apple Developer account. Takes 24-48 hours for App Store review.

```
# Install Capacitor
npm install @capacitor/core @capacitor/cli @capacitor/ios

# Initialize (run once per app)
npx cap init "App Name" com.yourcompany.appname --web-dir dist

# Add iOS platform
npx cap add ios

# Sync your web build to iOS
npx cap sync

# Open in Xcode for final build and submission
npx cap open ios
```

> **App Store submission requires Xcode on a Mac.** The review process takes 1-3 business days. Prepare screenshots at 1290x2796px (iPhone 15 Pro Max) and a 1024x1024px icon. These can be generated with Claude Code + a headless browser.

Domain Strategy

10 products on 10 subdomains from one $12/year domain. Surge and Vercel both support custom subdomains on free tiers.

```
# One domain. Multiple products. Zero extra cost.
calculator.yourdomain.com  -> surge dist/calculator/ calculator.yourdomain.com
tools.yourdomain.com       -> surge dist/tools/ tools.yourdomain.com
```

```
roi.yourdomain.com          -> surge dist/roi/ roi.yourdomain.com

# Point subdomain to Surge:
# Add CNAME record in DNS: subdomain -> na-west1.surge.sh
# Then deploy: surge dist/ subdomain.yourdomain.com
```

Deployment Checklist

Platform	What It Handles	What It Doesn't Handle
Surge.sh	Static files, custom domains, HTTPS, instant deploys	Server-side logic, databases, form submissions
Vercel	Full-stack apps, serverless functions, env variables, CI/CD	Long-running processes, files over 50MB
Gumroad	Payments, file delivery, VAT, refunds, affiliate program	Subscriptions (use Stripe), custom checkout UI
App Store	Distribution, payments (30% fee), updates, discovery	Web-only features, same-day deploys, sideloading

The 5-Question Non-Technical Review

You do not need to understand every line of code to catch 90% of real problems. These five questions cover what actually matters to a user.

The 5 Questions

1. Does it do what the spec says?

Go back to your CLAUDE.md. Read each bullet point under "What it does." Test each one manually. Check them off. If any bullet point fails, that is a revision item.

TEST FIRST

2. Does it break when I do something unexpected?

Try wrong inputs. Leave fields empty and click submit. Enter text where a number is expected. Enter a negative number. Enter a number with 10 decimal places. If it crashes or shows an error, that is a revision item.

EDGE CASES

3. Does it look right on mobile?

Resize your browser window to 375px wide (iPhone SE width). Does text overflow? Do buttons stack correctly? Is anything cut off or overlapping? If yes, that is a revision item.

MOBILE CHECK

4. Is there anything it does that I didn't ask for?

Scope creep check. Does it have features you didn't specify? Extra inputs, extra sections, extra buttons? If Claude Code added things you didn't ask for, note them. Keep what is useful, remove what isn't.

SCOPE CHECK

5. Can I explain what it does to a user in one sentence?

If you can't describe it in one sentence, users won't understand it either. Clarity check. If the answer is no, the product is either too complex or not focused enough. That is a spec problem, not a build problem.

CLARITY TEST

What to Do When You Find a Problem

Never say "it's broken." Describe three things:

```
1. WHAT YOU SEE: "When I click Calculate with the margin field empty,
   the output section shows 'NaN' instead of an error message."

2. WHAT YOU EXPECTED: "I expected it to show a red error message
   saying 'Please enter a margin percentage.'"

3. RELEVANT HTML SECTION: Paste the <div> or <section> that
   contains the broken element so Claude Code knows exactly where to look.
```

The "Show Me" Command

When you're not sure what Claude Code built or how it handles an edge case, ask directly:

> **"Show me exactly what happens when a user clicks Calculate with no data entered in any field."**
> Claude Code will trace through the logic and describe the output. You don't need to read the code. You just need to understand the behavior.

When to Accept Imperfect

> **If it does the job and doesn't break, ship it.**
> Perfect is the enemy of revenue. A working product with one minor visual quirk that earns $200/month beats a polished product you're still refining at month three. The revision cycle continues after shipping. Customers will tell you what actually matters.

Common Problems and Exact Revision Prompts

Problem You See	Exact Revision Prompt
Output shows "NaN" on empty input	"When any input field is empty, show a red error message instead of calculating. Do not show NaN or undefined."

Layout breaks on mobile (375px)	"The layout breaks at 375px width. Make the two-column layout stack vertically below 600px. Each section should take full width on mobile."
Button doesn't respond to click	"The Calculate button does not trigger any action when clicked. The click event should call the calculate() function. Check that the button has the correct id and the event listener targets it."
Numbers show too many decimals	"All currency outputs should show exactly 2 decimal places and a leading dollar sign. Use toFixed(2) and prepend '$'."
Page loads blank	"The page is blank when opened in a browser. Check for JavaScript syntax errors. Check that all DOM element IDs referenced in the script actually exist in the HTML."

The Product Catalog System

At 10 products, you can keep track in your head. At 30, you can't. At 50+, you need a system. This is the system.

The Master Catalog CSV

One file. Checked weekly. Every product in one place.

Column	What Goes Here	Example
Name	Product name	Agency Retainer Calculator
Type	HTML tool / Landing page / PDF / App / SaaS	HTML tool
File	Absolute path to the source file	PRODUCTS/agency_retainer_calc.html
URL	Live deploy URL	https://retainer.yourdomain.com
Price	Price in USD	$39

Platform	Where it's sold	Gumroad
Status	Idea / Spec / Built / Deployed / Live / Revenue / Kill	Revenue
MRR	Monthly recurring revenue (or monthly sales)	$312
Launch Date	Date first deployed	2026-02-14
Kill Date	Date to evaluate for kill trigger	2026-04-14

The Status System

Every product moves through these stages. The status column tells you what action to take next.

```
Idea       -> Write the spec. No action until spec exists.
Spec       -> Build it. Run: claude --dangerously-skip-permissions
Built      -> Run the 5-question review. Find and fix revision items.
Deployed   -> Submit to platform (Gumroad upload, Surge deploy, etc.)
Live       -> Market it. 3 tweets, 1 Reddit post, email list.
Revenue    -> Evaluate: Kill trigger or Double-down trigger.
Kill       -> Archive the file. Remove from active tracking.
```

Kill Trigger

> **Less than $100 revenue after 60 days live and marketed = Kill or Reprice.**
> "Marketed" means you actually promoted it: posted about it, mentioned it in your newsletter, submitted it to Product Hunt or relevant subreddits. If you built it and never told anyone, that's a distribution failure, not a product failure. Fix distribution first. If you marketed it and it still didn't sell in 60 days, kill it.

Double-Down Trigger

> **20%+ month-over-month growth at $500+ MRR = Double down.**
> When a product hits this threshold: add a Pro version, build variations for adjacent niches, increase content frequency about it, run a paid ad test at $5/day, reach out to newsletters for a sponsored mention. A product growing 20%/month at $500 MRR is on track for $6,000 MRR in 18 months with no additional builds.

Uptime Monitoring Without DevOps

Add this cron job to check that every deployed URL responds. Paste into terminal with `crontab -e`.

```
# Check all products weekly on Sunday at 8 AM
0 8 * * 0 python3 /path/to/project/check_uptime.py >> /path/to/project/uptime.log 2
```

The check_uptime.py script (build with Claude Code from this spec):

```
# check_uptime.py spec for Claude Code:
# Read CATALOG.csv. For each row where Status = Live or Revenue,
# send an HTTP GET to the URL in the URL column.
# If status code is not 200, write to uptime.log:
#   FAIL: [URL] returned [status_code] at [timestamp]
# If all pass, write: ALL OK at [timestamp]
# No external dependencies. Use only Python standard library.
```

The Pricing Model

Product Type	Recommended Price	PWYW Minimum	Why
HTML calculator tool (niche)	$39	$19	Specific enough to justify $39. PWYW minimum captures budget-constrained buyers.
PDF guide / checklist	$19–29	$9	Lower perceived value than interactive tools. Price to volume.

Swipe file / template pack	$29-47	$14	Specific templates command higher prices than generic ones.
PWA app (one-time)	$4.99-9.99	N/A	App Store pricing norms. One-time beats subscription for utility apps.
PWA app (subscription)	$2.99-4.99/mo	N/A	Only if you have ongoing content or features to justify recurring.
SaaS (subscription)	$29-99/mo	N/A	Annual option at 20% discount reduces churn. $29/year rarely works.

> **Why $39 beats $47 for info products:** $39 clears the "is this worth it" threshold faster. The price ends in 9 (psychological anchor). It's far enough below $47 to feel like a decision the buyer made, not a price they were pushed to. Tested across 20+ products. $39 with a PWYW minimum of $19 outperforms both $29 flat and $47 flat in total revenue per visitor.

Sample Catalog CSV Structure

```
Name,Type,File,URL,Price,Platform,Status,MRR,LaunchDate,KillDate
Agency Retainer Calc,HTML tool,PRODUCTS/retainer.html,retainer.yourdomain.com,$39,G
SaaS Pricing Calc,HTML tool,PRODUCTS/saas_pricing.html,pricing.yourdomain.com,$39,G
Founder Waitlist,Landing page,PRODUCTS/waitlist.html,waitlist.yourdomain.com,Free,S
Cold Email Pack,PDF,PRODUCTS/email_pack.pdf,,,$29,Gumroad,Built,,
Revenue Tracker App,App,apps/revenue-tracker/,,,$4.99,App Store,Spec,,
```

> **Weekly review takes 10 minutes:** Open CATALOG.csv. Check each Live product's revenue. Update MRR column. Apply kill or double-down triggers. Move any Built products to Deployed. That's it. The catalog is the system. The catalog is the business.

Built by a non-technical founder who shipped 50+ products without writing a single line of code from scratch. Not a course. Not a masterclass. Just the system.

Works offline. No tracking. No cookies. Open this file in any browser.